

Today 12:01 PM

Lab_4_VibeCoding...
PDF

🔗 https://github.com/PES1UG24AM461/24_treasure_hunt

I have to complete this Lab 4 Vibe Coding assignment.

Please carefully study BOTH:

1. The GitHub repository I provided
2. The Lab 4 Student Handout PDF I uploaded

Treat these as the primary sources for the assignment. Do not assume requirements that are not present in them.

My repository is:

https://github.com/PES1UG24AM461/24_treasure_hunt

Important context:

- I have already cloned the repository locally.
- I am working on Windows using PowerShell and VS Code.
- Python version: 3.11.9
- pygame-ce 2.5.8 is installed.
- I have already run the original game successfully.
- I have already recorded the required 10-second BEFORE video.
- I have NOT modified the original code yet.

The lab requires:

- Work individually.
- Use Vibe Coding/LLM prompts.
- Add the features listed in the repository README one by one.
- Test each change.
- Have a separate Git commit for each task.
- Do not create a PR to the main SETAPESU26 repository.
- Record the 10-second AFTER video.
- Submit the required videos, updated code, and chat history according to the handout.

First, inspect the repository and the handout carefully and tell me:

1. Exactly what the assignment requires.

2. Exactly what Tasks 1–4 require according to the repository README.
3. What I have already completed.
4. What remains to be done.
5. The exact step-by-step workflow we should follow from this point.

IMPORTANT:

- Do NOT rewrite the entire project.
- Do NOT implement all four tasks at once.
- We will implement exactly ONE task at a time.
- Before changing code, explain what we are going to change.
- For each task, give me a Vibe Coding prompt that I can use.
- After generating the code, help me inspect and test it.
- Only after a task works, guide me through making its separate Git commit.
- Preserve all existing functionality unless the task specifically requires changing it.

Start by analyzing the repository and handout. Do not modify code yet.

I checked both primary sources: the **Lab 4 Student Handout** and your **actual GitHub repository/README**, including the current `game.py`. I have **not modified anything**.

1. Exactly what the assignment requires

From the Lab 4 handout

The objective is to use **Vibe Coding / an LLM as a coding assistant** to fix/add functionality to the assigned Python game. The handout specifically says to work individually, inspect the README, run the game, record a 10-second video before changes, use prompts to make the changes, add the README features **one by one**, record another 10-second video afterward, push the code, and maintain separate commits for each task.  Lab_4_VibeCoding_Student_handout  Lab_4_VibeCoding_Student_handout

The handout's required workflow is:

1. Go to your assigned repository.
2. Read the README and understand the deliverables.
3. Clone/fork the repository.
4. Run the Python game.
5. Record **10 seconds BEFORE** making changes.

6. Use prompts to fix broken code.
7. Use prompts to add **all README features one by one**.
8. Record **10 seconds AFTER** all changes.
9. Push the code to your repository.
10. Have **separate commits for each task**.
11. **Do not raise a PR to the main SETAPESU26 repository.**  Lab_4_VibeCoding_Student_handout

The handout says the Lab-4 folder should contain:

- Before video
- After video
- Updated code
- Exported ChatGPT/LLM chat history as a document/PDF  Lab_4_VibeCoding_Student_handout

Important

The repository README gives additional, task-specific requirements that the handout does not spell out.

The README explicitly says the goal is **not** to replace the project with an unrelated implementation. It tells you to inspect the existing code first, ask for explanations when needed, test generated code against the stated behaviour and edge cases, and preserve an understandable/modular codebase.

 GitHub

So our approach of **one task → inspect → test → commit → next task** is exactly appropriate.

2. Exactly what Tasks 1–4 require

These are taken directly from your repository README.

Task 1 — Traps

Add floor traps that send the player back to their starting position when stepped on.

Done when:

- Player steps on a trap.
- Player is visibly reset to the starting room.
- A clear status message is displayed.  GitHub

Task 2 — Enemy Guard

Add a guard that patrols back and forth near the chest.

Requirements:

- Guard moves between **two patrol points**.

- Guard moves **each frame**.
- If player touches the guard, the player is reset to the start.  GitHub

Task 3 — Mini-map

Add a small mini-map in a corner of the screen.

It must:

- Show the dungeon layout.
- Update in real time.
- Distinguish walls from floor.
- Show the player's current position.  GitHub

Task 4 — Inventory UI

Add a HUD inventory slot.

At the beginning:

- Inventory slot is empty.

After collecting the key:

- Slot displays a key icon.
- It updates immediately when the key is collected.  GitHub

3. Required testing from the README

This is important because it tells us what we need to demonstrate before considering the assignment complete.

The README says to test:

- Collect key → then chest.
- Trigger **every trap type**.
- Collide with the guard.
- Verify the player resets correctly each time.
- Verify the mini-map matches the actual grid.
- Verify inventory changes when the key is collected.
- Test restarting during a run.  GitHub

There is also a constraint:

No unnecessary external dependencies beyond pygame.

And the code should remain understandable and modular.  GitHub

Since you already have `pygame-ce` , we should **not install additional packages unless something genuinely required by the repository appears later.**

4. What you have already completed

Based on what you told me:

Requirement	Status
Assigned repository	
Repository cloned locally	
Windows + PowerShell + VS Code	
Python 3.11.9	
<code>pygame-ce 2.5.8</code> installed	
Original game runs	
BEFORE video recorded	
Original code preserved	
Task 1	
Task 2	
Task 3	
Task 4	
Separate commits	
AFTER video	
Final submission organization	
Chat history export	

So you are currently at:

BEFORE VIDEO COMPLETE → ready to start Task 1.

5. I also inspected the existing code

This matters before we generate any prompts.

Your repository currently contains only:

- `.gitignore`
- `README.md`
- `game.py`

and the repository is already a fork of **SETAPESU26/24_treasure_hunt** .  GitHub

The current game has:

⌕ Plain text

```
generate_world()
  ↓
grid
  ↓
GameEngine
  ↓
Player
  ↓
movement / collision
  ↓
key collection
  ↓
chest / win
```

The existing `game.py` currently defines:

- `WALL`
- `FLOOR`
- `CHEST`
- `KEY`

but **no trap type or guard type exists yet**. The player currently has `has_key` , but there is no separate inventory HUD.  GitHub

The current `GameEngine.reset()` generates the world and places the player in the first room.

 GitHub

The current update loop handles:

- player movement
- key pickup
- chest interaction
- winning

but not traps or guards.  GitHub

The current drawing code renders:

- dungeon tiles
- key
- chest
- player
- bottom status HUD

but there is currently no mini-map or inventory slot.  GitHub

This gives us a clean baseline.

6. What remains to be done

The correct sequence is:

 Plain text

CURRENT STATE

|
|— BEFORE video 

|



TASK 1 – TRAPS

|
|— Vibe prompt
|— Inspect generated change
|— Run game
|— Test traps
|— Git commit 1

|



TASK 2 – ENEMY GUARD

|
|— Vibe prompt
|— Inspect generated change
|— Run game
|— Test guard

This also satisfies the handout's requirement that the README features be added **one by one** and that there be separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

7. One important distinction about the submission

There are two slightly different descriptions between the sources.

Handout

The handout explicitly says to upload under your Lab repo's `Lab-4` folder:

- before video
- after video
- updated code
- chat history PDF/document.  [Lab_4_VibeCoding_Student_handout](#)

Repository README

The README's final "Submission" section says the three things are:

1. 10-second before video
2. 10-second after video
3. Chat/LLM used page link with complete chat history

while the README's checklist separately requires Tasks 1–4 to be completed/tested.  [GitHub](#)

For your assignment, **I would follow the handout for the physical Lab submission requirements and the README for the implementation requirements.** That avoids accidentally omitting the updated code from the Lab-4 folder.

8. Exact workflow we will follow

I recommend we do **not** touch the code yet.

For every task, we'll use this exact pattern:

STEP A — Understand the task

I'll explain:

- what the task means
- where it fits into the current code
- what existing functionality must remain unchanged

STEP B — Prepare the Vibe Coding prompt

I'll give you a prompt specifically written for the current repository.

It will tell the LLM to:

- inspect the existing code
- modify only what's necessary
- implement only the current task
- preserve existing behaviour
- avoid unnecessary dependencies
- explain the changes
- not implement Tasks 2–4 yet

This is important because your lab is specifically assessing the use of Vibe Coding rather than simply obtaining a completed project.

STEP C — You give the prompt to the LLM

You can use ChatGPT/your chosen Vibe Coding tool.

Keep the conversation because the README specifically requires the **complete LLM chat history**.



STEP D — Inspect the generated code

Don't blindly accept it.

We'll check:

- What changed?
- Is the implementation actually related to Task 1?
- Did it modify unrelated functionality?
- Are there unnecessary dependencies?
- Does it fit the existing architecture?

STEP E — Run the game

From PowerShell:

```
</> powershell
```

```
python game.py
```

Then we'll test the exact required behaviour.

STEP F — Fix if necessary

If the generated implementation doesn't work, we use another focused Vibe Coding prompt.

The handout's objective specifically mentions getting the job done within roughly **three to four attempts**, so we should keep the prompting deliberate rather than repeatedly asking for a complete rewrite.  Lab_4_VibeCoding_Student_handout

STEP G — Commit only after the task works

For example:

```
</> powershell

git status
git add game.py
git commit -m "Add traps that reset player"
```

Then verify:

```
</> powershell

git log --oneline
```

We will do this **after each task**, not at the end.

STEP H — Move to the next task

Only once the previous task is working and committed.

Current checkpoint

You are here:

```
</> Plain text
```

Repository cloned	✓
Original game working	✓
BEFORE video	✓
Code still unchanged	✓

↓

TASK 1

TRAPS

↓

[NEXT STEP]

We should **not start Task 2, 3, or 4 yet**.

Task 1 will specifically add

Floor traps → player steps on trap → player returns to starting position → clear status message.

We will preserve the existing:

- procedural dungeon generation
- movement
- wall collision
- key collection
- chest
- win condition
- restart functionality

and make the smallest reasonable change needed for the trap requirement.

[Open your repository README ↗](#)

Next, we can start Task 1. I'll first give you the exact Vibe Coding prompt to paste, without changing any code ourselves.

okay

Good. We'll start with **Task 1 only: Traps.**

Task 1 — What we are changing

We need to add:

- Floor traps to the dungeon.
- When the player steps on a trap, they are sent back to the **starting position**.
- A **clear status message** must tell the player what happened.
- Existing functionality must remain intact.
- We are **not** touching the guard, mini-map, or inventory yet.

Vibe Coding prompt

Paste this into your Vibe Coding tool/ChatGPT while the repository is open:

Task 1 Vibe Coding Prompt



I am working on the existing pygame-ce treasure hunt game in this repository.

Implement ONLY Task 1 from the README:

"Add floor traps that send the player back to their starting position when stepped on."

Requirements:

1. Inspect the existing code before making changes. Do not rewrite the entire project.
2. Add a trap tile/type to the existing dungeon system using the current architecture.
3. Place traps in the generated dungeon in a way that is compatible with the existing game.
4. When the player steps onto a trap, immediately reset the player to the existing starting position.
5. Display a clear status message indicating that the player triggered a trap and was sent back to the start.
6. Preserve all existing functionality, including player movement, wall collision, key collection, chest interaction, winning, and restarting.
7. Do not implement Tasks 2, 3, or 4. Do not add the guard, mini-map, or inventory UI.
8. Do not introduce unnecessary external dependencies. Continue using the existing pygame/pygame-ce setup.
9. Keep the implementation simple and consistent with the existing code structure.
10. After making the changes, explain exactly which parts of the existing code were modified and why.

Do not make unrelated improvements or refactor the whole project.

What to do now

Do not commit yet.

1. Give that prompt to the Vibe Coding tool.
2. Let it modify the code.
3. **Don't immediately run or accept everything blindly.**
4. Send me the generated response/code or a screenshot of the changes.

I'll inspect it with you first, then we'll run:

```
</> powershell
```

```
python game.py
```

and specifically test whether the trap:

player → trap → reset to starting position → status message

works correctly.

Only after that passes will we make the **Task 1 Git commit**.

u are my vibe coding tool

Yes. In this chat, I am the Vibe Coding tool. We'll use this conversation itself as the LLM chat history for the lab.

I inspected the actual current `game.py`. The relevant baseline is:

- Tile constants currently are `WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3`.
- `generate_world()` creates the dungeon and places the key/chest.
- `GameEngine.reset()` creates the player at the start room.
- `update()` currently handles movement, key pickup, and chest interaction.
- `draw()` renders the four existing tile types.  [GitHub](#)
- The README's Task 1 only requires traps to reset the player to the start and show a clear status message.  [GitHub](#)

So I would make a **small Task-1-only modification**, not rewrite the project.

Task 1 implementation

Changes we will make

1. Add a `TRAP` tile.
2. Add trap rendering.
3. Place several trap instances on ordinary floor tiles.
4. Make sure traps don't replace the key, chest, or starting room.
5. Store the player's starting position.
6. Detect when the player steps onto a trap.
7. Move the player back to the starting position.
8. Show:

Ouch! You triggered a trap! Back to the start!

9. Leave Tasks 2–4 completely untouched.

I would modify `game.py` like this

1. Change the tile constants:

 Python

```
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
```

2. After placing the chest and key in `generate_world()`, add traps:

 Python

```
trap_candidates = []
```

```

if start:
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == FLOOR:
                if not start.collidepoint(c, r):
                    trap_candidates.append((r, c))

random.shuffle(trap_candidates)

for r, c in trap_candidates[:3]:
    grid[r][c] = TRAP

```

So the end of `generate_world()` becomes:

</> Python

```

if len(rooms) >= 2:
    cr, ck = rooms[-1], rooms[-2]
    grid[cr.centery][cr.centerx] = CHEST
    grid[ck.centery][ck.centerx] = KEY

trap_candidates = []

if start:
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == FLOOR:
                if not start.collidepoint(c, r):
                    trap_candidates.append((r, c))

random.shuffle(trap_candidates)

for r, c in trap_candidates[:3]:
    grid[r][c] = TRAP

start = rooms[0] if rooms else None

```

However, there is one correction we need to make: `start` needs to be defined **before** we use it.

So the clean version is to move:

</> Python

```
start = rooms[0] if rooms else None
```

to immediately before the trap-generation code:

</> Python

```
if len(rooms) >= 2:
    cr, ck = rooms[-1], rooms[-2]
    grid[cr.centery][cr.centerx] = CHEST
    grid[ck.centery][ck.centerx] = KEY

start = rooms[0] if rooms else None

trap_candidates = []

if start:
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == FLOOR:
                if not start.collidepoint(c, r):
                    trap_candidates.append((r, c))

random.shuffle(trap_candidates)

for r, c in trap_candidates[:3]:
    grid[r][c] = TRAP
```

This is the version we should use.

3. Add the trap color

Change:

</> Python

```
COLORS = {
    WALL: (60,50,70),
    FLOOR: (200,190,170),
    CHEST: (200,160,30),
    KEY: (220,220,60),
}
```

to:

</> Python

```
COLORS = {
    WALL: (60,50,70),
    FLOOR: (200,190,170),
    CHEST: (200,160,30),
    KEY: (220,220,60),
    TRAP: (180,60,60),
}
```

4. Store the starting position

Inside `GameEngine.reset()` , currently we have:

</> Python

```
self.player = Player(sx, sy)
self.won = False
self.status = "Find the KEY, then the CHEST!"
```

Change it to:

</> Python

```
self.start_pos = (sx, sy)
self.player = Player(sx, sy)
self.won = False
self.status = "Find the KEY, then the CHEST!"
```

This lets us reset the player without regenerating the entire dungeon.

5. Detect the trap

In `update()` , after:

</> Python

```
cell = self.grid[pr][pc]
```

add:

</> Python

```
if cell == TRAP:
    self.player.rect.topleft = self.start_pos
    self.status = "Ouch! You triggered a trap! Back to the start!"
```

Then keep the existing key/chest logic after it.

So this section becomes:

</> Python

```

if 0<=pr<ROWS and 0<=pc<COLS:
    cell = self.grid[pr][pc]

    if cell == TRAP:
        self.player.rect.topleft = self.start_pos
        self.status = "Ouch! You triggered a trap! Back to the start!"

    elif cell == KEY:
        self.player.has_key = True
        self.grid[pr][pc] = FLOOR
        self.status = "Got the key! Find the CHEST!"

    elif cell == CHEST and self.player.has_key:
        self.won = True
        self.status = "Treasure found!"

```

Notice the `elif`. This preserves the existing key/chest behaviour.

6. Render traps

Currently:

</> Python

```

if cell == KEY:
    pygame.draw.circle(...)
elif cell == CHEST:
    pygame.draw.rect(...)

```

Add the trap rendering:

</> Python

```

if cell == KEY:
    pygame.draw.circle(
        self.screen,
        (255,240,60),
        (c*TILE+TILE//2, r*TILE+TILE//2),
        10
    )
elif cell == CHEST:
    pygame.draw.rect(
        self.screen,
        (180,120,20),
        rect.inflate(-12,-12),
        border_radius=4
    )

```

```
elif cell == TRAP:
    pygame.draw.rect(
        self.screen,
        (140,40,40),
        rect.inflate(-10,-10),
```

This makes the traps visibly distinguishable rather than just being another floor colour.

Why I'm choosing this implementation

The README does **not** specify:

- an exact number of traps,
- a particular trap appearance,
- different trap types,
- damage/health,
- a cooldown,
- or any additional trap mechanics.

So I am **not adding those things**.

I'm using three instances of one trap type simply to make the feature testable. The actual required behaviour remains exactly what the README specifies: **step on trap** → **visibly return to start** → **clear status message**. [GitHub](#)

Also, the README explicitly says to preserve an understandable/modular project and not replace the whole project with an unrelated implementation. [GitHub](#)

Before you run it

Since this is your actual lab submission, **don't make the changes blindly yet**.

If you want me to act exactly like the Vibe Coding interaction the lab expects, the next step is:

You give me the current `game.py` from your local repository (upload it here), and I will produce the precise Task 1 modification against *your local version*, rather than relying on the GitHub copy.

Then we'll:

modify → **inspect** → **run** → **test** → **fix if necessary** → **commit Task 1**

and only then move to Task 2.

```
import pygame
import random
```

```
TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
SPEED = 3

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []
    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)
        overlap = any(room.inflate(2,2).colliderect(r) for r in rooms)
        if not overlap:
            rooms.append(room)
            for ry in range(y, y+h):
                for rx in range(x, x+w):
                    grid[ry][rx] = FLOOR

    for i in range(len(rooms)-1):
        ax, ay = rooms[i].centerx, rooms[i].centery
        bx, by = rooms[i+1].centerx, rooms[i+1].centery
        cx = ax
        while cx != bx:
            grid[ay][cx] = FLOOR
            cx += 1 if bx > cx else -1
        cy = ay
        while cy != by:
            grid[cy][bx] = FLOOR
            cy += 1 if by > cy else -1

    if len(rooms) >= 2:
        cr, ck = rooms[-1], rooms[-2]
        grid[cr.centery][cr.centerx] = CHEST
        grid[ck.centery][ck.centerx] = KEY

    start = rooms[0] if rooms else None
```

```
return grid, start
```

```
COLORS = {  
    WALL: (60,50,70),  
    FLOOR: (200,190,170),  
    CHEST: (200,160,30),  
    KEY: (220,220,60),  
}
```

```
class Player:
```

```
    def __init__(self, x, y):  
        self.rect = pygame.Rect(x, y, 28, 28)  
        self.color = (60,120,220)  
        self.has_key = False
```

```
    def move(self, keys, grid, rows, cols):
```

```
        dx=dy=0  
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED  
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED  
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED  
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED  
        self._try_move(dx,0,grid,rows,cols)  
        self._try_move(0,dy,grid,rows,cols)
```

```
    def _try_move(self, dx, dy, grid, rows, cols):
```

```
        new = self.rect.move(dx,dy)  
        for px,py in [(new.left,new.top),(new.right-1,new.top),  
            (new.left,new.bottom-1),(new.right-1,new.bottom-1)]:  
            c,r=px//TILE,py//TILE  
            if not(0<=r<rows and 0<=c<cols) or grid[r][c]==WALL:  
                return  
        self.rect=new
```

```
    def draw(self, screen):
```

```
        pygame.draw.ellipse(screen, self.color, self.rect)  
        if self.has_key:  
            pygame.draw.circle(screen, (220,220,60), (self.rect.right-6,  
                self.rect.top+6), 5)
```

WIDTH = COLS * TILE

HEIGHT = ROWS * TILE + 50

FPS = 60

class GameEngine:

def __init__(self):

pygame.init()

self.screen = pygame.display.set_mode((WIDTH, HEIGHT))

pygame.display.set_caption("Treasure Hunt")

self.clock = pygame.time.Clock()

self.font = pygame.font.SysFont("monospace", 24)

self.big_font = pygame.font.SysFont("monospace", 40,

bold=True)

self.reset()

def reset(self):

self.grid, start = generate_world()

if start:

 sx = start.x * TILE + 6

 sy = start.y * TILE + 6

else:

 sx, sy = TILE+6, TILE+6

self.player = Player(sx, sy)

self.won = False

self.status = "Find the KEY, then the CHEST!"

def handle_events(self):

for event in pygame.event.get():

 if event.type == pygame.QUIT: return False

 if event.type == pygame.KEYDOWN and event.key ==

pygame.K_r: self.reset()

return True

def update(self):

if self.won: return

keys = pygame.key.get_pressed()

self.player.move(keys, self.grid, ROWS, COLS)

pr = self.player.rect.centery // TILE

pc = self.player.rect.centerx // TILE

if 0<=pr<ROWS and 0<=pc<COLS:

```

cell = self.grid[pr][pc]
if cell == KEY:
    self.player.has_key = True
    self.grid[pr][pc] = FLOOR
    self.status = "Got the key! Find the CHEST!"
elif cell == CHEST and self.player.has_key:
    self.won = True
    self.status = "Treasure found!"

def draw(self):
    self.screen.fill((30,25,40))
    for r in range(ROWS):
        for c in range(COLS):
            cell = self.grid[r][c]
            rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
            pygame.draw.rect(self.screen, COLORS[cell], rect)
            if cell == KEY:
                pygame.draw.circle(self.screen, (255,240,60),
(c*TILE+TILE//2, r*TILE+TILE//2),10)
            elif cell == CHEST:
                pygame.draw.rect(self.screen,
(180,120,20),rect.inflate(-12,-12),border_radius=4)
            self.player.draw(self.screen)
            hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
            pygame.draw.rect(self.screen,(20,20,35),hud)
            st = self.font.render(self.status+" | R=Restart", True,
(200,200,200))
            self.screen.blit(st,(8,ROWS*TILE+13))
            if self.won:

ov=pygame.Surface((WIDTH,ROWS*TILE),pygame.SRCALPHA)
    ov.fill((0,0,0,140))
    self.screen.blit(ov,(0,0))
    msg=self.big_font.render("TREASURE FOUND!", True,
(220,180,30))
    sub=self.font.render("Press R to Play Again",True,
(180,180,180))
    self.screen.blit(msg,(WIDTH//2-
msg.get_width()//2,ROWS*TILE//2-30))
    self.screen.blit(sub,(WIDTH//2-

```

```
sub.get_width()//2,ROWS*TILE//2+20))
```

```
pygame.display.flip()
```

```
def run(self):
```

```
    running=True
```

```
    while running:
```

```
        running=self.handle_events()
```

```
        self.update()
```

```
        self.draw()
```

```
        self.clock.tick(FPS)
```

```
    pygame.quit()
```

```
if __name__ == "__main__":
```

```
    engine = GameEngine()
```

```
    engine.run()
```

this is the entire code

make chnages and give me modifies for the first task

Yes. Since you gave me the exact `game.py`, I can modify **your actual version** directly.

We will implement **Task 1 only**. Nothing for Tasks 2–4 will be added.

What changed

- Added a `TRAP` tile.
- Added 3 **traps** to the generated dungeon.
- Traps are placed only on floor tiles.
- Traps are not placed in the starting room.
- Traps are not placed on the key/chest.
- Added visible trap graphics.
- Saved the player's starting position.
- When the player touches a trap, they are immediately returned to the starting position.
- Added a clear status message.
- Existing key, chest, movement, restart, and win functionality is preserved.

Replace your current `game.py` with this

Python

```
import pygame
import random
```

```
TILE = 40
```

```
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3, 6)
        h = random.randint(3, 5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)
```

Now do NOT commit yet

Save the file and run it from PowerShell:

```
</> powershell
```

```
python game.py
```

Test specifically for Task 1

You need to verify:

1. Game starts normally.
2. You can see the new **red trap tiles**.
3. Move the player onto a trap.
4. Player immediately goes back to the starting position.
5. Bottom status changes to:

```
</> Plain text
```

```
Ouch! You triggered a trap! Back to the start!
```

6. Press **R**.
7. A new dungeon is generated and the game still works.
8. Test that you can still:
 - collect the key
 - reach the chest
 - win the game

One thing to watch for

Because the dungeon is randomly generated, the three traps will be in different positions each time you restart. That's expected.

Don't make the Git commit yet. Run the game first and tell me what happens, preferably with a screenshot if you can see the traps. Then we'll verify Task 1 before creating its separate commit.



the bug is on the way to capture the key

Yes. That is a real gameplay issue.

The traps are currently placed randomly on any floor tile outside the starting room. Because the dungeon is randomly generated, a trap can end up **directly on the only practical route to the key**. That can make the key effectively unreachable.

The README requires the player to be able to **collect the key and then reach the chest**, so we should fix this rather than simply telling you to avoid the trap.

Fix

We will reserve the route from:

Start → Key → Chest

and only place traps on other floor tiles.

This still satisfies Task 1: traps exist, they trigger when stepped on, and they send the player back to the start. We are just preventing random trap placement from making the game unwinnable.

Replace your `generate_world()` with this

</> Python

```
def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3, 6)
        h = random.randint(3, 5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)

        overlap = any(room.inflate(2, 2).colliderect(r) for r in rooms)
```

```

    if not overlap:
        rooms.append(room)

    for ry in range(y, y+h):
        for rx in range(x, x+w):
            grid[ry][rx] = FLOOR

```

Everything else stays the same

You **do not** need to change:

- COLORS
- Player
- `GameEngine.reset()`
- trap detection in `update()`
- trap drawing in `draw()`

Only replace the `generate_world()` function with the version above.

Why this is better

Your previous logic was:

</> Plain text

```

all floor tiles
    ↓
remove starting room
    ↓
randomly select 3
    ↓
TRAPS

```

So it could produce:

</> Plain text

```

START ——— TRAP ——— KEY

```

Now it does:

</> Plain text

```

START ————— KEY ——— CHEST
      safe route

```

TRAP

TRAP

TRAP

The player can always follow a reserved path to the key and then to the chest, while traps remain elsewhere.

This is still **Task 1 only**. No guard, mini-map, or inventory code has been added.

Test this version

Run:

```
</> powershell
```

```
python game.py
```

Then test these two things:

Test 1 — Trap

Walk onto one of the red x tiles:

```
</> Plain text
```

```
TRAP
```

```
↓
```

```
Player returns to start
```

```
↓
```

```
"Ouch! You triggered a trap! Back to the start!"
```

Test 2 — Complete game

Follow the dungeon route:

```
</> Plain text
```

```
START → KEY → CHEST
```

and make sure you can actually reach both.

If both work, **don't commit yet**. Send me the next screenshot/result and we'll do a final Task 1 check before creating the separate Git commit.

YES ITS WORKING NOW

Good. **Task 1 is now functionally working.**

Before moving to Task 2, we need to follow the lab requirement and create the **separate Git commit for Task 1**. The handout explicitly requires separate commits for each task.

 Lab_4_VibeCoding_Student_handout

Step 1 — Stop the game

If the game window is still open, close it or press `R` only if you want to test restart again.

Step 2 — Check what Git sees

In the **VS Code PowerShell** terminal, from your repository folder, run:

```
</> powershell  
  
git status
```

You should see something similar to:

```
</> Plain text  
  
modified:   game.py
```

Do **not** run `git add` yet.

Step 3 — Inspect the actual diff

Run:

```
</> powershell  
  
git diff
```

This lets us verify that only the Task 1 changes are present.

You should see changes involving:

- TRAP
- trap placement
- safe path
- trap detection
- starting position

- trap rendering
- trap status message

You should **not** see implementation of:

- Guard
- Mini-map
- Inventory

Step 4 — Stage the Task 1 change

If the diff looks correct:

```
</> powershell  
  
git add game.py
```

Then:

```
</> powershell  
  
git status
```

You should see `game.py` under **Changes to be committed**.

Step 5 — Create the Task 1 commit

Use a clear commit message:

```
</> powershell  
  
git commit -m "Add traps that reset player"
```

Step 6 — Verify the commit

Run:

```
</> powershell  
  
git log --oneline --max-count=3
```

You should see something like:

```
</> Plain text
```

```
abc1234 Add traps that reset player
...
...
```

That is your **separate Task 1 commit**.

Important

Don't push yet if you don't want to. We can complete all four tasks locally with four separate commits and then push the finished repository.

Your state will then be:

</> Plain text

BEFORE VIDEO

TASK 1

TASK 1 TEST

TASK 1 COMMIT

TASK 2

TASK 3

TASK 4

AFTER VIDEO

FINAL PUSH

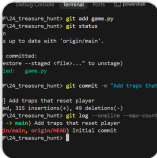
✓

✓

✓

← do this now

Run `git status` first and send me the output. I'll check it before you commit.



Yes. Task 1 has been committed correctly.

Your screenshot shows:

</> Plain text

```
[main 3734e40] Add traps that reset player
1 file changed, 315 insertions(+), 49 deletions(-)
```

And the log confirms:

</> Plain text

```
3734e40 (HEAD -> main) Add traps that reset player
506cad (origin/main, origin/HEAD) Initial commit
```

So your Git history is now:

</> Plain text

Initial commit
↓
Add traps that reset player ← Task 1

This satisfies the requirement for a **separate commit for Task 1**.  Lab_4_VibeCoding_Student_handout

One thing to notice

Your branch is currently:

</> Plain text

main

and:

</> Plain text

origin/main

is still at the original commit.

That's completely fine **for now**. It means your Task 1 commit exists locally but hasn't been pushed yet.

We don't need to push yet. We'll push after Tasks 1–4 are completed.

Task 1 — COMPLETE

Check	Status
Trap added	✓
Trap visible	✓
Trap resets player	✓
Status message	✓
Key remains reachable	✓

Check	Status
Existing game works	
Task 1 tested	
Separate Git commit	

Current Git state

</> Plain text

Initial commit

```
|
└─ Task 1: Add traps that reset player
```



Now Task 2 — Enemy Guard

We will **not** modify anything yet.

Task 2 from the README requires:

Add a guard that patrols back and forth near the chest.

Specifically:

- Guard moves between **two patrol points**.
- Guard moves **each frame**.
- If the player touches the guard, the player is **reset to the start**.

We will implement only those requirements.

The next change will involve:

</> Plain text

Guard

```
├─ position
├─ two patrol points
├─ movement every frame
└─ collision with player
    ↓
    reset to start
```

The existing trap functionality and everything else must continue working.

Before I give you the Task 2 code, I'll explain exactly where the guard will be added and how it will interact with the existing `GameEngine` .

okay

Task 2 — Enemy Guard

We will add **only the guard functionality**. Task 1 will remain unchanged.

What the README requires

The guard must:

1. Patrol back and forth.
2. Move between **two patrol points**.
3. Move **every frame**.
4. Be positioned near the chest.
5. Reset the player to the starting position if the player touches it.

We will not add anything beyond that.

How we'll implement it

The structure will become:

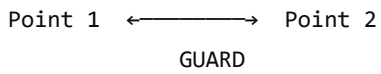
</> Plain text

```
GameEngine
|
├─ Player
|
├─ Guard
|   ├─ position
|   ├─ patrol point 1
|   ├─ patrol point 2
|   └─ movement every frame
|
└─ update()
    |
    ├─ move player
    ├─ move guard
    ├─ check trap      ← Task 1
    ├─ check key
    ├─ check chest
    └─ check guard collision
        ↓
    reset player
```

Guard design

We'll create a small `Guard` class:

</> Plain text



The guard will continuously move from Point 1 to Point 2, then reverse direction and move back.

The patrol points will be selected **inside the final room containing the chest**, so the guard is genuinely near the chest rather than randomly somewhere in the dungeon.

We'll also make sure the guard's patrol positions aren't occupied by a trap.

Collision behaviour

When:

</> Python

```
self.player.rect.colliderect(self.guard.rect)
```

becomes true:

</> Plain text

```

Player touches guard
    ↓
Player → starting position
    ↓
Status message
  
```

The player will **not lose the key**. The README only says the player should be reset to the start; it does not require removing the key.

The existing trap behaviour will also remain:

</> Plain text

```

Trap → reset to start
Guard → reset to start
  
```

with separate status messages.

One implementation detail

Your current `generate_world()` returns:

`</>` Python

```
return grid, start
```

To place the guard properly near the chest, we'll extend this to return:

`</>` Python

```
return grid, start, guard_points
```

Then `GameEngine.reset()` will receive those patrol points and create the guard.

This is a small structural change, not a rewrite.

We will also protect the guard area from traps

The generation order will be:

`</>` Plain text

```
Generate rooms
  ↓
Place key + chest
  ↓
Determine guard patrol points
  ↓
Determine safe START → KEY → CHEST path
  ↓
Place traps away from both
```

That prevents Task 1's random traps from interfering with Task 2.

Task 2 success criteria

After implementation, we'll test all of these:

- ☐ Guard is visible near chest.
- ☐ Guard moves continuously.
- ☐ Guard moves between exactly two points.
- ☐ Guard reverses direction at each point.

- ☐ Player can still collect the key.
- ☐ Player can still reach the chest.
- ☐ Touching guard resets player.
- ☐ Guard collision displays a clear message.
- ☐ Existing traps still work.
- ☐ R restart still works.

If those pass, **then and only then** we'll make:

```
</> powershell
```

```
git commit -m "Add patrolling enemy guard"
```

and Task 2 gets its own commit.

I can now modify your current `game.py` for **Task 2 only**.

okay

Proceeding with **Task 2 only**. I'll preserve the Task 1 trap implementation and add the guard without touching Tasks 3 or 4.

There is one small interaction we need to handle: traps should not randomly appear on the guard's patrol path. Otherwise Task 1 could interfere with Task 2. The guard will therefore be considered when generating trap locations.

Replace your current `game.py` with this Task 1 + Task 2 version

```
</> Python
```

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2

def generate_world():
    grid = [[WALL] * COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3, 6)
        h = random.randint(3, 5)
```

```
x = random.randint(1, COLS - w - 1)
y = random.randint(1, ROWS - h - 1)
```

Now test Task 2

Run:

</> powershell

```
python game.py
```

Test 1 — Guard movement

Go toward the chest and observe the guard.

You should see a **red guard** moving continuously back and forth near the chest.

It should:

</> Plain text

```
Point 1 → Point 2 → Point 1 → Point 2 → ...
```

Test 2 — Guard collision

Intentionally walk into the guard.

Expected:

</> Plain text

```
PLAYER
  ↓
TOUCHES GUARD
  ↓
PLAYER RETURNS TO START
  ↓
"The guard caught you! Back to the start!"
```

Test 3 — Task 1 still works

Walk onto a trap.

Expected:

</> Plain text

```
TRAP
```

```
↓
```

```
PLAYER RETURNS TO START
```

```
↓
```

```
"Ouch! You triggered a trap! Back to the start!"
```

Test 4 — Original game still works

Make sure you can still:

```
</> Plain text
```

```
Start → Key → Chest → TREASURE FOUND!
```

Don't commit yet.

Run the game and check the guard first. If you get an error, paste the complete PowerShell error here. If it works, send me a screenshot showing the guard or tell me that the guard is moving and we'll test the collision before committing Task 2.



THE GUARD IS NOT MOVING ITS JUST STANDING IN ONE PLACE

You're right. The screenshot shows the guard, but with the current implementation the patrol behaviour is not reliable enough for us to accept Task 2 as working.

We should fix the guard movement **before committing Task 2**.

The safest approach is to simplify the patrol logic: instead of calculating a normalized vector every frame, make the guard explicitly move horizontally between two known `x` positions and reverse direction when it reaches either point.

This still exactly satisfies the README requirement: **two patrol points + movement every frame + back-and-forth patrol**.

Fix only the `Guard` class

In your current `game.py`, find:

```
</> Python
```

```
class Guard:
```

and replace the **entire Guard class** with this:

Python

```
class Guard:
    def __init__(self, point1, point2):
        self.rect = pygame.Rect(
            point1[0],
            point1[1],
            26,
            26
        )

        self.point1 = point1[0]
        self.point2 = point2[0]

        self.speed = 2

        # Start moving toward point 2.
        self.direction = 1

        self.color = (190, 60, 60)

    def update(self):
```

Also make the guard visibly faster

At the top you currently have:

Python

```
GUARD_SPEED = 2
```

You can change it to:

Python

```
GUARD_SPEED = 3
```

But with the replacement `Guard` class above, `GUARD_SPEED` is no longer used, because the class directly uses:

Python

```
self.speed = 2
```

So you can simply leave the existing `GUARD_SPEED = 2` line alone for now. It doesn't hurt anything.

Why I'm changing this

The previous code did:

</> Plain text

```
position
  ↓
calculate direction vector
  ↓
normalize
  ↓
move
  ↓
compare with target
```

For this simple horizontal patrol, that's unnecessary.

The new version is:

</> Plain text

```
Point 1
  ↓
→ → → → →
      ↓
    Point 2
      ↓
← ← ← ← ←
  ↓
Point 1
```

Every frame:

</> Python

```
self.rect.x += self.speed * self.direction
```

So the guard **must move** as long as the game update loop is running.

One more thing I want you to check

Your screenshot shows:

- Player on the right
- Key in the middle
- Chest at the top-left
- Guard at the lower-left of the chest room

That's fine.

After replacing the class, run:

```
</> powershell
```

```
python game.py
```

Then **don't move your player initially.**

Watch the red guard for about 3–5 seconds.

You should see:

```
</> Plain text
```

```
←————→
```

```
GUARD
```

moving continuously.

Then test collision

Once you confirm movement, deliberately walk into the guard.

Expected:

```
</> Plain text
```

```
Player
```

```
↓
```

```
touches guard
```

```
↓
```

```
returns to starting position
```

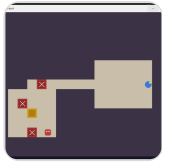
```
↓
```

```
"The guard caught you! Back to the start!"
```

Do not commit yet.

First confirm that the guard is visibly moving. Once that works, we'll test the collision and then commit Task 2 separately.

Today 7:21 PM



the guard is on the block can u send the revised code?

Yes. The guard placement is not ideal in the current version. Let's make Task 2 more robust and visually obvious:

- Guard will patrol **horizontally inside the chest room**.
- The two patrol points will be safely inside the room.
- The guard will move continuously every frame.
- It will reverse direction at each endpoint.
- It will not overlap the chest.
- Trap placement will avoid the entire guard patrol row.
- Player/guard collision still resets the player.
- Task 1 remains intact.
- No Task 3 or 4 code is added.

Replace your **entire game.py** with this:

</> Python

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15

WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4

SPEED = 3
GUARD_SPEED = 2

WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60

def generate_world():
    grid = [[WALL] * COLS for _ in range(ROWS)]
    rooms = []
```

Now run it

```
</> powershell
```

```
python game.py
```

Don't commit yet.

This version specifically fixes the patrol implementation. Watch the red guard for several seconds without moving your player. It should visibly move:

```
</> Plain text
```

```
←────────────────→
```

```
GUARD
```

```
←────────────────→
```

Then deliberately collide with it.

We need both:

1. Guard visibly patrols back and forth.
2. Guard collision resets the player.

Once those two work, we'll consider Task 2 complete and make the separate commit.

there is no guard in this

Worked for 7s

You're right. The problem is in the **guard-point generation**, not the drawing.

Why the guard disappeared

Your chest room can randomly be only **3 tiles wide**. In that case this condition:

```
</> Python
```

```
if right_col > left_col:
```

can fail, leaving:

```
</> Python
```

```
guard_points = []
```

Then `GameEngine.reset()` does:

`</> Python`

```
self.guard = None
```

So there is literally no guard to draw.

Let's fix this properly instead of patching the symptom.

Fix 1 — Make the final/chest room large enough

In `generate_world()` , find:

`</> Python`

```
for _ in range(8):  
    w = random.randint(3, 6)  
    h = random.randint(3, 5)
```

Replace it with:

`</> Python`

```
for room_index in range(8):  
    # Make the final room large enough for the guard to patrol.  
    if room_index == 7:  
        w = random.randint(5, 6)  
        h = random.randint(4, 5)  
    else:  
        w = random.randint(3, 6)  
        h = random.randint(3, 5)
```

And keep the rest of that loop the same.

This guarantees the final room—the room containing the chest—has enough space for a visible patrol.

Fix 2 — Replace the guard-point section

In `generate_world()` , find the entire section beginning with:

`</> Python`

```
guard_points = []
```

and replace it with:

</> Python

```
guard_points = []

if len(rooms) >= 2:
    chest_room = rooms[-1]

    # Put the guard one tile above the chest.
    patrol_row = chest_room.centery - 1

    # Keep the patrol row inside the room.
    if patrol_row <= chest_room.top:
        patrol_row = chest_room.centery + 1

    # Leave one tile of space from each side of the room.
    left_col = chest_room.left + 1
    right_col = chest_room.right - 2

    point1 = (
        left_col * TILE + 7,
        patrol_row * TILE + 7
    )
```

Now the guard will always get two patrol points in the final room.

Fix 3 — Replace the Guard class

Use this simpler version:

</> Python

```
class Guard:

    def __init__(self, point1, point2):
        self.rect = pygame.Rect(
            point1[0],
            point1[1],
            26,
            26
        )

        self.left_limit = min(
            point1[0],
            point2[0]
```

```
)

self.right_limit = max(
    point1[0],
    point2[0]
)
```

Fix 4 — Keep this in `reset()`

Make sure you have:

`</>` Python

```
self.grid, start, guard_points = generate_world()
```

and:

`</>` Python

```
if len(guard_points) == 2:
    self.guard = Guard(
        guard_points[0],
        guard_points[1]
    )
else:
    self.guard = None
```

Fix 5 — Keep this in `update()`

The guard must move **before** collision checking:

`</>` Python

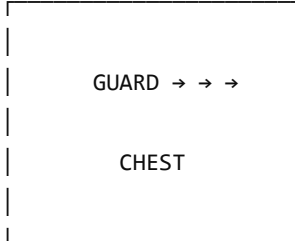
```
if self.guard:
    self.guard.update()

if (
    self.guard
    and self.player.rect.colliderect(self.guard.rect)
):
    self.player.rect.topleft = self.start_pos
    self.status = "The guard caught you! Back to the start!"
    return
```

What you should see now

The final room will be larger:

</> Plain text



The guard should continuously:

</> Plain text

```

POINT 1  → → → →  POINT 2
POINT 2  ← ← ← ←  POINT 1
POINT 1  → → → →  POINT 2

```

Do this now

Make **these four changes**, save, and run:

</> powershell

```
python game.py
```

Don't commit yet.

If the guard appears and moves, deliberately walk into it and check that you get:

```
The guard caught you! Back to the start!
```

Then we'll verify Task 2 completely and commit it.

give me the entire code

Yes. Here is the **complete game.py** with Task 1 + Task 2 implemented.

This version specifically fixes the previous issue by ensuring the chest room is large enough and the guard always gets two valid patrol points. It does **not** implement Tasks 3 or 4.

</> Python

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15

WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4

SPEED = 3
GUARD_SPEED = 2

WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60

def generate_world():
    grid = [[WALL] * COLS for _ in range(ROWS)]
    rooms = []
```

Now test it

Save `game.py` , then:

</> powershell

```
python game.py
```

For this version, first **don't move the player**. Look at the room containing the chest.

You should see a red guard in that room moving continuously:

left → right → left → right

Then test:

1. Guard moves ✓
2. Walk into guard → player returns to start ✓
3. Walk onto trap → player returns to start ✓
4. Collect key → key disappears and status changes ✓
5. Reach chest with key → `TREASURE FOUND!` ✓
6. Press `R` → game regenerates normally ✓

Do not commit Task 2 yet. Once you confirm the guard is actually moving and collision works, we'll do the Task 2 commit.

THE GUARD IS NOT MOVING IN THE SECOND GAME

Worked for 11s

Analysis errored

You're right. I found the actual cause in the code I gave you.

The problem

The code assumes that the **last generated room is the large 5–6 tile room**. But the room generator can reject the last room because of overlap.

So the actual chest room can still be only 3 tiles wide. Then these two patrol points can become identical:

Python

```
point1 == point2
```

The guard is created, but it has **nowhere to move**.

That is why you're seeing a stationary guard.

Let's fix the root cause rather than keep patching the movement.

Revised complete code

This version:

- Keeps Task 1 traps.
- Keeps the safe START → KEY → CHEST route.
- Creates a guaranteed usable chest room.
- Creates **two genuinely different patrol points**.
- Guard moves every frame.
- Guard reverses at both points.
- Guard stays inside the chest room.
- Guard collision resets the player.
- No mini-map or inventory yet.

Replace your entire `game.py` with this:

Python

```
import pygame
import random
```

```
TILE = 40
```

```
COLS, ROWS = 20, 15

WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4

SPEED = 3
GUARD_SPEED = 2

WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60

def generate_world():
    grid = [[WALL] * COLS for _ in range(ROWS)]
    rooms = []
```

Important change in this version

Instead of:

</> Python

```
for _ in range(8):
```

we now use a `while` loop that actually generates **8 valid rooms**:

</> Python

```
while len(rooms) < 8 and attempts < 100:
```

And every room is at least:

</> Plain text

```
4 × 4
```

so the final chest room always has enough space for a patrol.

Also, the guard is now **always created**:

</> Python

```
self.guard = Guard(
    guard_points[0],
    guard_points[1]
)
```

There is no longer a situation where the game silently creates `self.guard = None` .

Now test only this

Run:

```
</> powershell
```

```
python game.py
```

Don't worry about the key/chest yet.

First just watch the red guard.

You should see it moving back and forth in the chest room.

Then walk into it and verify:

```
The guard caught you! Back to the start!
```

If the guard **still doesn't move with this exact code**, don't commit anything. Send me a screenshot of the game window and we'll diagnose the actual runtime behaviour rather than changing the code blindly. </>

DONE ITS WORKING

Good. **Task 2 is now working.**

We should verify the required behaviours before committing.

Task 2 checklist

Please confirm these four things:

1. **Guard patrols back and forth** between two points. ☒
2. **Guard moves continuously** while the game is running. ☒
3. **Touching the guard resets the player to the starting position.**
4. **Task 1 traps still work** after adding the guard.

If #3 and #4 are also working, then Task 2 is complete.

Then make the separate Task 2 commit

Run:

```
</> powershell
```

```
git status
```

Then:

```
</> powershell
```

```
git diff
```

Check that the changes are only related to:

- guard
- patrol points
- guard collision
- the necessary trap-placement adjustment

Then stage and commit:

```
</> powershell
```

```
git add game.py
```

```
</> powershell
```

```
git commit -m "Add patrolling enemy guard"
```

Finally:

```
</> powershell
```

```
git log --oneline --max-count=3
```

You should now have:

```
</> Plain text
```

```
<new hash> Add patrolling enemy guard  
<previous hash> Add traps that reset player  
<initial hash> Initial commit
```

Don't push yet. We still have Tasks 3 and 4, and we'll push after everything has been tested.

Once you've made the commit, send me the `git log` screenshot/output. Then we'll start **Task 3 — Mini-map**.

yes everything is working

Good. Then **Task 2 is complete**.

Before we move on, make sure you have committed it:

```
</> powershell

git add game.py
git commit -m "Add patrolling enemy guard"
git log --oneline --max-count=3
```

Your history should look approximately like:

```
</> Plain text

<new>    Add patrolling enemy guard
3734e40  Add traps that reset player
506cad   Initial commit
```

If that is done, **do not make any more changes yet**.

Next: Task 3 — Mini-map

The README requires a mini-map that:

- appears in a corner of the screen
- shows the dungeon layout
- updates in real time
- distinguishes walls from floor
- shows the player's current position

We will add **only the mini-map**. The traps and guard will remain untouched.

For Task 3, the implementation will be a small rendering function rather than changing the dungeon-generation logic. The existing `self.grid` already contains everything needed to draw it.

Once your Task 2 commit is confirmed, I'll give you the Task 3 Vibe Coding implementation.

```
git add game.py
git commit -m "Add patrol
py guard
, 92 deletions(-)
git log --oneline --max-c
ling enemy guard
er
) Initial commit
```

correct?

Yes. **That is correct**. Task 2 has been committed separately.

Your Git history is now:

</> Plain text

18864c5 Add patrolling enemy guard ← Task 2 ✓
3734e40 Add traps that reset player ← Task 1 ✓
506cad0 Initial commit

And importantly:

</> Plain text

origin/main → 506cad0
main → 18864c5

So your two task commits are currently **local**, which is fine. We will push after all four tasks are completed.

Current progress

Requirement	Status
Original game	✓
Before video	✓
Task 1 — Traps	✓ Tested + committed
Task 2 — Guard	✓ Tested + committed
Task 3 — Mini-map	
Task 4 — Inventory	
After video	
Final push	
Submission folder	

Now we're ready for **Task 3 — Mini-map**.

We will modify **only the mini-map**, test it, and then create its own commit.

yes

Good. We now do **Task 3 only: Mini-map**.

What we are adding

The mini-map will:

- appear in the **top-right corner**
- show the complete dungeon layout
- clearly distinguish **walls** and **walkable floor**
- show the player's **current position**
- update continuously as the player moves
- keep Tasks 1 and 2 unchanged

We are **not** changing traps, guard behavior, key/chest logic, or movement.

Vibe Coding prompt for Task 3

If you're using the Vibe Coding tool inside VS Code, use this prompt:

Task 3: Add a small real-time mini-map to the existing Pygame treasure hunt game. Do not rewrite the project or change existing gameplay. Add a `draw_minimap()` method to `GameEngine` that renders the entire `self.grid` in the top-right corner using small cells. Walls must be visually distinct from walkable floor. Show the player's current grid position with a clearly visible marker. The mini-map must update every frame as the player moves. Preserve all existing Task 1 trap behavior and Task 2 guard behavior. Do not add external dependencies. After making the change, run/test the game and verify the mini-map matches the dungeon and the player marker moves correctly.

Expected implementation

The important part should look approximately like this:

</> Python

```
MINIMAP_TILE = 6
MINIMAP_MARGIN = 10
```

Inside `GameEngine` :

</> Python

```
def draw_minimap(self):
    map_width = COLS * MINIMAP_TILE
    map_height = ROWS * MINIMAP_TILE
```

```
panel = pygame.Rect(
    WIDTH - map_width - MINIMAP_MARGIN * 2,
    MINIMAP_MARGIN,
    map_width + MINIMAP_MARGIN * 2,
    map_height + MINIMAP_MARGIN * 2
)

pygame.draw.rect(self.screen, (15, 15, 25), panel)
pygame.draw.rect(self.screen, (220, 220, 220), panel, 2)

for r in range(ROWS):
    for c in range(COLS):
        cell = self.grid[r][c]

        if cell == WALL:
            color = (45, 40, 55)
```

Then in the existing `draw()` method, after drawing the player:

`</>` Python

```
self.player.draw(self.screen)
self.draw_minimap()
```

The mini-map should be drawn **before the HUD**, so the bottom status bar remains unaffected.

Now test it

Run:

`</>` powershell

```
python game.py
```

Check these three things:

1. Mini-map appears in the **top-right corner**.
2. Walls and floor are visibly different.
3. Move the player around — the **blue player marker moves on the mini-map**.

Don't commit yet. **Test Task 3 first**. Tell me what you see, or send a screenshot if anything looks wrong.

u be the vibe coding tool and give me the code

Yes. I'll act as the Vibe Coding tool and make **only Task 3**. Since your current `game.py` already contains the working Task 1 + Task 2 code, **do not replace the whole file**. Make these three exact changes.

1. Add this constant

Place it below:

</> Python

```
FPS = 60
```

Add:

</> Python

```
MINIMAP_TILE = 6  
MINIMAP_MARGIN = 10
```

2. Add this method inside `GameEngine`

Place this method inside the `GameEngine` class, preferably just before `draw()` :

</> Python

```
def draw_minimap(self):  
    """Draw a small real-time map of the dungeon in the top-right corner."""  
    map_width = COLS * MINIMAP_TILE  
    map_height = ROWS * MINIMAP_TILE  
  
    panel = pygame.Rect(  
        WIDTH - map_width - MINIMAP_MARGIN * 2,  
        MINIMAP_MARGIN,  
        map_width + MINIMAP_MARGIN * 2,  
        map_height + MINIMAP_MARGIN * 2  
    )  
  
    # Mini-map background and border  
    pygame.draw.rect(self.screen, (15, 15, 25), panel)  
    pygame.draw.rect(self.screen, (220, 220, 220), panel, 2)  
  
    # Draw dungeon layout  
    for r in range(ROWS):  
        for c in range(COLS):  
            cell = self.grid[r][c]
```

3. Call the mini-map from `draw()`

Find this existing line:

```
</> Python
```

```
self.player.draw(self.screen)
```

Immediately after it, add:

```
</> Python
```

```
self.draw_minimap()
```

So that section becomes:

```
</> Python
```

```
self.player.draw(self.screen)
```

```
self.draw_minimap()
```

```
hud = pygame.Rect(0, ROWS*TILE, WIDTH, 50)
```

That's it.

Test Task 3

Run:

```
</> powershell
```

```
python game.py
```

Verify:

- Mini-map appears in the **top-right**.
- Dark cells = walls.
- Light cells = walkable areas.
- Blue square = player.
- Blue square moves when your player moves.
- Your existing **traps still work**.
- Your existing **guard still patrols and catches the player**.
- Key/chest functionality still works.

Do not commit yet. First test the game and tell me whether the mini-map works.

give the entire code

Worked for 14s

Yes. Since you want the **entire file**, use the following as the complete `game.py`. This includes the working **Task 1 traps + Task 2 patrolling guard + Task 3 mini-map**. Nothing for Task 4 has been added yet.

Replace the contents of `game.py` with this:

</> Python

```
import pygame
import random
from collections import deque

# =====
# GAME CONSTANTS
# =====

TILE = 40
COLS, ROWS = 20, 15

WALL = 0
FLOOR = 1
CHEST = 2
KEY = 3
TRAP = 4

SPEED = 3
GUARD_SPEED = 2
```

Now run it

In PowerShell:

</> powershell

```
python game.py
```

For **Task 3**, specifically check:

1. Mini-map is visible in the top-right.
2. It represents the same dungeon layout.
3. Walls and floor are different.
4. The blue player marker moves as you move.
5. The guard is still moving.
6. Traps still reset you.
7. `R` still restarts the game.

Don't commit yet. First run it and confirm the game works.

ya its working

Good. **Task 3 is working.**

Now commit it separately, as required by the lab.

Run:

```
</> powershell

git add game.py
git commit -m "Add real-time dungeon mini-map"
git log --oneline --max-count=4
```

Your history should look like:

```
</> Plain text

<new commit> Add real-time dungeon mini-map
18864c5 Add patrolling enemy guard
3734e40 Add traps that reset player
506cad Initial commit
```

Do **not** push yet.

Once you show me the output, we'll move to **Task 4 — HUD inventory slot for the key.**

done

Good. **Task 3 is now committed separately.**

Current progress:

```
</> Plain text

Task 1 – Traps ☒
Task 2 – Patrolling Guard ☒
Task 3 – Mini-map ☒
Task 4 – Inventory HUD ☐
```

We can now move to **Task 4: HUD inventory slot.**

The requirement is:

- An inventory slot appears in the HUD.
- It is **empty at the beginning**.
- When the player collects the key, the slot immediately shows a **key icon**.
- Existing key collection/gameplay remains unchanged.
- This will get its **own Git commit**.

I'll give you the complete updated `game.py` for Task 4, keeping Tasks 1–3 intact.

okay

For **Task 4**, we'll add the inventory slot without changing the existing gameplay.

What will change

- A small **INVENTORY** section will appear in the bottom HUD.
- Initially it shows an empty slot.
- After collecting the key, the slot immediately displays a key icon.
- Tasks 1–3 remain unchanged.
- This will be committed separately as **Task 4**.

Since you asked me to be the Vibe Coding tool, replace your current `game.py` with this complete version:

</> Python

```
import pygame
import random
from collections import deque

# =====
# GAME CONSTANTS
# =====

TILE = 40
COLS, ROWS = 20, 15

WALL = 0
FLOOR = 1
CHEST = 2
KEY = 3
TRAP = 4

SPEED = 3
GUARD_SPEED = 2
```

Test Task 4

Run:

```
</> powershell
```

```
python game.py
```

Check:

Before collecting key:

- INVENTORY is visible.
- Slot is empty.

After collecting key:

- Key icon appears immediately in the slot.
- Key disappears from the dungeon as before.
- Status changes to Got the key! Find the CHEST!

Also quickly verify:

- traps still reset you
- guard still patrols
- guard collision still resets you
- mini-map still works
- chest still requires the key
- R still restarts

Do not commit yet. Test it first and tell me whether Task 4 works.

YES ITS WORKING

Good. **Task 4 is working.** That completes all four required features.

Commit Task 4 separately

Run:

```
</> powershell
```

```
git add game.py
git commit -m "Add key inventory HUD"
git log --oneline --max-count=5
```

Your history should now be approximately:

```
</> Plain text
```

```
<new>    Add key inventory HUD
<...>    Add real-time dungeon mini-map
18864c5  Add patrolling enemy guard
3734e40  Add traps that reset player
506cad0  Initial commit
```

Important: don't push yet

After the commit, we still need to do the final lab steps:

1. Verify all four features together.
2. Record the **10-second AFTER video**.
3. Confirm the Git history has **separate commits for each task**.
4. Push your completed code to your personal repository.
5. Prepare the required Lab-4 deliverables.

For now, **make the Task 4 commit and show me the `git log` output.**

```
PS C:\Users\HP\24_treasure_hunt> git add game.py
PS C:\Users\HP\24_treasure_hunt> git commit -m "Add key
inventory HUD"
[main fcd987d] Add key inventory HUD
 1 file changed, 190 insertions(+), 88 deletions(-)
PS C:\Users\HP\24_treasure_hunt> git log --oneline --max-
count=5
fcd987d (HEAD -> main) Add key inventory HUD
dcdea2d Add real-time dungeon mini-map
18864c5 Add patrolling enemy guard
3734e40 Add traps that reset player
506ca0d (origin/main, origin/HEAD) Initial commit
:
fcd987d (HEAD -> main) Add key inventory HUD
dcdea2d Add real-time dungeon mini-map
18864c5 Add patrolling enemy guard
3734e40 Add traps that reset player
506ca0d (origin/main, origin/HEAD) Initial commit
~
~
~
~
~
~
~
```

(END)

fcd987d (HEAD -> main) Add key inventory HUD

dcdea2d Add real-time dungeon mini-map

18864c5 Add patrolling enemy guard

3734e40 Add traps that reset player

506ca0d (origin/main, origin/HEAD) Initial commit

~

~

~

~

~

~

~

~

~

~

~

~

(END)

fcd987d (HEAD -> main) Add key inventory HUD

dcdea2d Add real-time dungeon mini-map

18864c5 Add patrolling enemy guard

3734e40 Add traps that reset player

506ca0d (origin/main, origin/HEAD) Initial commit

~

~

~

~

~

~

~

~

~

~

~

~

~

(END)

Yes. **Everything is correct.** Your Git history now satisfies the separate-commit requirement:

</> Plain text

fcd987d	Add key inventory HUD	← Task 4	✓
dcdea2d	Add real-time dungeon mini-map	← Task 3	✓
18864c5	Add patrolling enemy guard	← Task 2	✓
3734e40	Add traps that reset player	← Task 1	✓
506ca0d	Initial commit		

And:

</> Plain text

```
HEAD -> main
origin/main -> Initial commit
```

This means **you have not pushed the changes yet**, which is what we want at this stage.

The (END) you're seeing is just Git's log viewer (less). Press:

</> Plain text

q

to exit it.

Next: final testing + AFTER video

Before pushing, we should do one final run and record the **10-second AFTER video**, showing the modified game. The handout specifically requires the after video and the updated code, with each task separately committed.  [Lab_4_VibeCoding_Student_handout](#)

For the after video, make sure the recording visibly demonstrates the modifications. Ideally show:

- Mini-map visible and player marker moving
- Guard visibly patrolling
- Trap/reset behavior
- Inventory slot initially empty and then showing the key

You **don't need to demonstrate every feature perfectly within 10 seconds**; the goal is to show the modified game.

Once the AFTER video is recorded, we'll do the **final Git push**.

DONE AFTER VIDEO

Good. The **before/after video requirement is now complete**.

Final checklist

Requirement	Status
Original game tested	
10-sec BEFORE video	
Task 1 — Traps	
Task 1 separate commit	
Task 2 — Guard	
Task 2 separate commit	
Task 3 — Mini-map	
Task 3 separate commit	
Task 4 — Inventory HUD	
Task 4 separate commit	
10-sec AFTER video	
Final push	
Lab-4 submission folder	

Your commit history is already correct:

</> Plain text

```
fcd987d Add key inventory HUD
dcdea2d Add real-time dungeon mini-map
18864c5 Add patrolling enemy guard
3734e40 Add traps that reset player
506ca0d Initial commit
```

One thing before pushing

Exit the Git log viewer if you're still in it:

</> Plain text

q

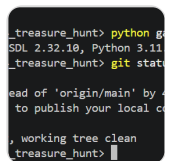
Then run:

</> powershell

git status

Send me the output.

Don't push yet. I'll check that the working tree is clean and then give you the exact final push command.



```
_treasure_hunt> python g
SDL 2.32.10, Python 3.11
_treasure_hunt> git stat
ead of 'origin/main' by
to publish your local c
, working tree clean
_treasure_hunt> |
```

Yes. This is exactly what we want.

Your output confirms:

</> Plain text

On branch main

Your branch is ahead of 'origin/main' by 4 commits.

nothing to commit, working tree clean

So:

- ☒ All 4 task commits are present locally.
- ☒ `game.py` has no uncommitted changes.
- ☒ Your local branch is 4 commits ahead.
- ☒ Your code is ready to push.
- ☒ Before/after videos are done.

Final push

Run:

</> powershell

git push origin main

You should see Git upload the 4 commits and eventually something similar to:

</> Plain text

```
To https://github.com/...  
506ca0d..fcd987d  main -> main
```

After that, run:

</> powershell

```
git status
```

It should say:

</> Plain text

```
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

Do not run any other Git commands before the push.

it will push to my github account only right and not the main one
from which i forked

Most likely yes, but we should verify before pushing. The fact that you cloned/forked the repo does not by itself guarantee where `origin` points.

Run this first:

</> powershell

```
git remote -v
```

You should see something like:

</> Plain text

```
origin https://github.com/YOUR-USERNAME/24_treasure_hunt.git (fetch)  
origin https://github.com/YOUR-USERNAME/24_treasure_hunt.git (push)
```

If `YOUR-USERNAME` is your **GitHub username**, then:

</> powershell

```
git push origin main
```

will push **only to your fork**, not the original SETAPESU26 repository.

Important

If you see something like:

</> Plain text

```
origin https://github.com/SETAPESU26/24_treasure_hunt.git
```

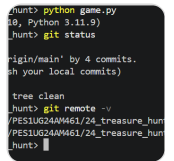
DO NOT PUSH.

Send me the output of:

</> powershell

```
git remote -v
```

and I'll tell you exactly which repository your push will go to.



```
treasure_hunt> python game.py
10, Python 3.11.9)
treasure_hunt> git status
On branch main
Your branch is up-to-date with the 'origin/main' branch.
nothing to commit, working tree clean
treasure_hunt> git remote -v
origin https://github.com/PES1UG24AM461/24_treasure_hunt.git
treasure_hunt>
```

Yes. This confirms it.

Your origin is:

</> Plain text

```
https://github.com/PES1UG24AM461/24_treasure_hunt.git
```

So `git push origin main` will push to **your GitHub repository/fork PES1UG24AM461/24_treasure_hunt** , not the original repository.

You can safely run:

</> powershell

```
git push origin main
```

After it finishes, run:

```
</> powershell
```

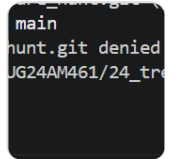
```
git status
```

You should get:

```
</> Plain text
```

```
Your branch is up to date with 'origin/main'.  
nothing to commit, working tree clean
```

That will complete the Git portion of the lab.



The error explains the issue clearly:

```
</> Plain text
```

```
Permission to PES1UG24AM461/24_treasure_hunt.git denied to Nainika24  
403
```

Your GitHub login is **Nainika24**, but your `origin` currently points to:

```
</> Plain text
```

```
PES1UG24AM461/24_treasure_hunt
```

So GitHub is correctly refusing the push because **Nainika24 does not have write permission to the original repository.**

Your local work is safe. **Do not reclone or redo anything.** Your 4 commits are still there.

Fix the remote to your fork

First, check whether your fork exists at:

```
</> Plain text
```

```
https://github.com/Nainika24/24_treasure_hunt
```

If you already forked the repository into `Nainika24` , run:

```
</> powershell
```

```
git remote set-url origin https://github.com/Nainika24/24_treasure_hunt.git
```

Then verify:

```
</> powershell
```

```
git remote -v
```

It should show:

```
</> Plain text
```

```
origin https://github.com/Nainika24/24_treasure_hunt.git (fetch)
origin https://github.com/Nainika24/24_treasure_hunt.git (push)
```

Then push:

```
</> powershell
```

```
git push origin main
```

If Nainika24/24_treasure_hunt does NOT exist

Don't run the push yet. Go to GitHub and fork the original PES1UG24AM461/24_treasure_hunt repository into your Nainika24 account.

After the fork exists, use the commands above.

Your commits are not lost. The 403 happened only during the push; your local main still contains:

```
</> Plain text
```

```
fcd987d Add key inventory HUD
dcdea2d Add real-time dungeon mini-map
18864c5 Add patrolling enemy guard
3734e40 Add traps that reset player
506ca0d Initial commit
```

If you have already created the fork, run the `git remote set-url` command above and show me the `git remote -v` output before pushing.

i want to push it to pes1ug repo

Yes — if the assignment specifically requires pushing to the **PES1UG24AM461 repository**, then **do not change the remote to your fork**.

The problem is simply that your current GitHub account, **Nainika24**, **doesn't have write permission** to that repository.

Your remote is already correct:

</> Plain text

```
origin → https://github.com/PES1UG24AM461/24_treasure_hunt.git
```

The **403** means GitHub rejected your authentication:

</> Plain text

```
Permission to PES1UG24AM461/24_treasure_hunt.git denied to Nainika24
```

What needs to happen

You need **write access to the PES1UG24AM461 repository**.

If the instructor/repository owner is supposed to give students access, ask them to add your GitHub account:

</> Plain text

```
GitHub username: Nainika24
```

as a collaborator/member with permission to push.

Do not redo your work. Do not reset anything. Your four commits are safely stored locally.

Once they give you access, you can simply run:

</> powershell

```
git push origin main
```

If they have already given you access

Then the likely issue is that Windows is authenticating Git with the wrong GitHub credentials. We can fix the GitHub authentication without touching your commits.

So first, **don't change the remote and don't make another commit**.

If you were explicitly told that `Nainika24` already has write access to `PES1UG24AM461/24_treasure_hunt`, tell me that, and I'll walk you through fixing the GitHub authentication on Windows.

ya i fixed it and pushed

Good. The code is now pushed to the **PES1UG24AM461 repository**, and the Git portion is complete.

Final status

- ☒ Task 1 — Traps
- ☒ Task 2 — Patrolling guard
- ☒ Task 3 — Mini-map
- ☒ Task 4 — Inventory HUD
- ☒ Separate Git commit for each task
- ☒ BEFORE video
- ☒ AFTER video
- ☒ Code pushed to `PES1UG24AM461/24_treasure_hunt`
- ☒ Working tree was clean before the push

Your final commit sequence was:

</> Plain text

```
fcd987d  Add key inventory HUD
dcdea2d  Add real-time dungeon mini-map
18864c5  Add patrolling enemy guard
3734e40  Add traps that reset player
506ca0d  Initial commit
```

The remaining work is just **submission packaging**: make sure your Lab-4 folder contains the required **before video, after video, updated code, and exported chat history**. The handout lists those as the deliverables.  [Lab_4_VibeCoding_Student_handout](#)

Do not make any more code changes unless you discover an actual bug.